

Wireless Drive-By-Wire

Design Document

Capstone Spring 2026

ECE 4773 — Senior Design

Document Version: 1.0
Date: May 2026
Audience: Future developers, maintainers

Note

This document is intended for engineers who need to **repair, modify, or extend** the system. It covers the internal hardware design, signal mappings, control algorithm, network protocol, and known design limitations. For operator instructions, see the separate *User Manual*.

0 Contents

1	System Overview	2
1.1	Top-Level Block Diagram	2
1.2	File Map	2
2	Hardware Design	2
2.1	Custom PCB	2
2.2	ESP32 Pin Assignments	3
2.3	MCP2515 CAN Controller	3
2.4	MCP4728 DAC Configuration	3
3	Signal Mapping and Baseline Voltages	4
3.1	DAC Channel Assignments	4
3.2	Steering Output (Push-Pull)	4
3.3	Throttle Output	5
3.4	Brake Output	5
3.5	All Outputs Clamped	5
4	CAN Bus Interface	5
4.1	Steering Angle Message	5
4.2	Reading Strategy	6
5	Steering PID Control Loop	6
5.1	Loop Structure	6
5.2	Target Calculation and Smoothing	6
5.3	PID Gains	7
5.4	Anti-Windup and Clamping	7
5.5	PID Tuning Guide	7
6	Network Architecture	8
6.1	Why a Relay Server?	8
6.2	Relay Server Logic	8
6.3	Packet Protocol	8
6.4	Network Configuration	9
7	Software Architecture	9
7.1	g29_to_server_original.py (Operator Laptop)	9
7.2	relay_server.py (OCI Cloud VM)	10
7.3	esp32_car.ino (ESP32 on PCB)	10
8	Known Limitations and Design Decisions	11
8.1	No Watchdog / Failsafe	11
8.2	Open-Loop Throttle and Brake	11
8.3	EEPROM Initialization Mismatch	12
8.4	DEBUG Prints in relay_server.py	12

8.5	Relay Server Has No Timeout on Car Address	12
9	Modification Guide	12
9.1	Changing WiFi Credentials	12
9.2	Changing the Relay Server IP or Port	12
9.3	Adjusting Throttle and Brake Scaling	13
9.4	Reading Additional CAN Signals	13
9.5	Adding the Watchdog	13

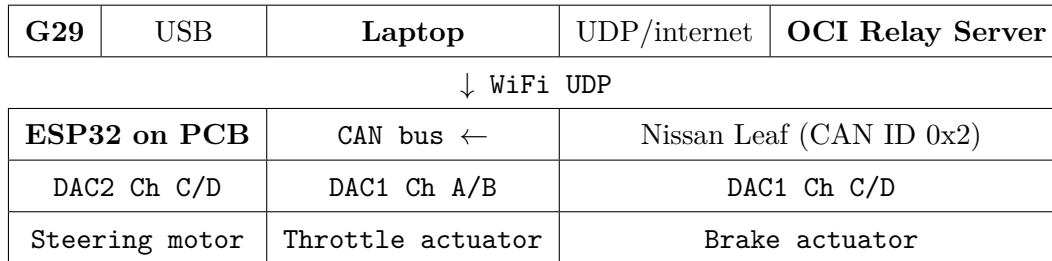
1 System Overview

The Wireless Drive-By-Wire (DBW) system replaces the mechanical inputs of a Nissan Leaf (steering wheel, throttle, brake pedal) with electronically generated analog voltages that the vehicle's ECU interprets as driver input.

An operator uses a Logitech G29 racing wheel and pedals connected to a laptop. Control packets are transmitted over UDP through an Oracle Cloud relay server to an ESP32 microcontroller mounted on a custom PCB inside the vehicle. The ESP32 reads the actual steering angle from the CAN bus and drives three systems:

- **Steering** — closed-loop PID, DAC voltages drive the power steering motor
- **Throttle** — open-loop, DAC voltages increase from a baseline
- **Brake** — open-loop, DAC voltages decrease/increase from a baseline

1.1 Top-Level Block Diagram



1.2 File Map

File	Runs on	Role
g29_to_server_original.py	Operator laptop	Read G29, send UDP
relay_server.py	OCI cloud VM	Forward UDP packets
esp32_car.ino	ESP32 on PCB	Control actuators

2 Hardware Design

2.1 Custom PCB

The custom DBW PCB integrates the following onto a single board:

- ESP32-WROOM-32 DevKit V1 (footprint / header)
- MCP2515 CAN bus controller (SPI) with 16 MHz crystal
- Two MCP4728 quad 12-bit I²C DACs
- Supporting passives (decoupling capacitors, pull-up resistors for I²C)

The PCB was fabricated by JLCPCB (2-layer).

2.2 ESP32 Pin Assignments

ESP32 Pin	Signal	Interface	Connected to
18	SPI SCK	SPI	MCP2515 SCK
19	SPI MISO	SPI	MCP2515 SO
23	SPI MOSI	SPI	MCP2515 SI
5	CAN CS	SPI	MCP2515 $\overline{\text{CS}}$
4	CAN INT	GPIO	MCP2515 INT (active-low)
21	I2C1 SDA	I ² C bus 0	DAC 1 (MCP4728 @ 0x60)
22	I2C1 SCL	I ² C bus 0	DAC 1 (MCP4728 @ 0x60)
33	I2C2 SDA	I ² C bus 1	DAC 2 (MCP4728 @ 0x60)
32	I2C2 SCL	I ² C bus 1	DAC 2 (MCP4728 @ 0x60)

Note

Both DACs share the same I²C address (0x60). They are disambiguated by being placed on *separate* I²C buses: W1 (TwoWire instance 0) and W2 (TwoWire instance 1). If you ever replace a DAC, verify that its address pins (A0, A1, A2) are all grounded so it enumerates as 0x60.

2.3 MCP2515 CAN Controller

The MCP2515 communicates with the ESP32 over SPI and with the vehicle over the OBD-II CAN bus.

Parameter	Value
CAN bus speed	500 kbps (CAN_500KBPS)
Crystal frequency	16 MHz (MCP_16MHZ)
Operating mode	Normal (MCP_NORMAL)
Filter	None (MCP_ANY — all IDs accepted)
INT pin polarity	Active-low; triggers on any received frame

△ WARNING

The firmware uses MCP_16MHZ. If the PCB crystal is ever replaced, it **must** be 16 MHz, not 8 MHz. A mismatch silently corrupts CAN timing; the symptom is that `currentSTA` never updates and steering drives to its limit.

2.4 MCP4728 DAC Configuration

Both DACs are configured identically: internal voltage reference with 2× gain.

Parameter	Setting	Effect
Voltage reference	MCP4728_VREF_INTERNAL	Uses 2.048 V internal ref
Gain	MCP4728_GAIN_2X	Full scale = $2 \times 2.048 = 4.096$ V
Resolution	12-bit	4096 steps, ≈ 1 mV/step

These settings are written in `initDACChannels()` and saved to the DACs' internal EEPROM via `saveToEEPROM()` so they survive power cycling.

2.4.0 Converting Voltage to DAC Count

$$\text{DAC count} = \left\lfloor \frac{V_{\text{target}}}{4.096} \times 4095 \right\rfloor$$

The firmware stores the reference voltage as `float proper = 4.096` and applies this formula in `updateBaseVoltages()` every loop iteration.

3 Signal Mapping and Baseline Voltages

3.1 DAC Channel Assignments

Variable	DAC	Ch	Function	Baseline V	DAC count
<code>steer1</code>	DAC 2	C	Steering main	2.47 V	2473
<code>steer2</code>	DAC 2	D	Steering sub	2.47 V	2473
<code>accel1Base</code>	DAC 1	A	Throttle main	0.37 V	370
<code>accel2Base</code>	DAC 1	B	Throttle sub	0.75 V	750
<code>brake1Base</code>	DAC 1	C	Brake main	3.42 V	3417
<code>brake2Base</code>	DAC 1	D	Brake sub	1.51 V	1509

Note

`updateBaseVoltages()` is called once per loop. Although the values are constant, keeping it in the loop makes them easy to change to live-tunable variables in the future (e.g., read from a serial command).

3.2 Steering Output (Push-Pull)

The steering motor is driven push-pull: one DAC channel raises above midpoint while the other drops by the same amount. The motor controller interprets the voltage *difference* as torque magnitude and direction.

DAC 2, Ch C = `steer1 + torque`

DAC 2, Ch D = `steer2 - torque`

At zero torque both channels output 2.47 V and the motor is stationary. `torque` is clamped to ± 1500 DAC counts ($\approx \pm 1.5$ V swing).

3.3 Throttle Output

$$\text{accelMag} = \lfloor \text{throttle} \times 300 \rfloor \quad (0\text{--}300)$$

$$\text{DAC 1, Ch A} = \text{accel1Base} + \text{accelMag}$$

$$\text{DAC 1, Ch B} = \text{accel2Base} + 2 \times \text{accelMag}$$

Channel B receives twice the offset of channel A. This mimics the dual-channel pedal sensor redundancy scheme used by the Nissan Leaf’s original pedal assembly (primary and secondary sensors track each other with a 2:1 voltage ratio).

3.4 Brake Output

$$\text{brakeMag} = \lfloor \text{brake} \times 900 \rfloor \quad (0\text{--}900)$$

$$\text{DAC 1, Ch C} = \text{brake1Base} - \text{brakeMag}$$

$$\text{DAC 1, Ch D} = \text{brake2Base} + \text{brakeMag}$$

△ WARNING

The brake channels must **never** be written to 0 V. The vehicle ECU monitors both brake sensor voltages; if either reads 0 V it interprets this as a broken wire (sensor fault) and may disable throttle or engage a failsafe. The baseline voltages (3.42 V and 1.51 V) are the “no braking” idle state, not 0 V.

3.5 All Outputs Clamped

Every DAC output is passed through `constrain(value, 0, 4095)` inside `sendOutputs()` before being written. This prevents integer overflow from wrapping a count above 4095 back to a small positive value.

4 CAN Bus Interface

4.1 Steering Angle Message

The Nissan Leaf broadcasts the steering angle on CAN ID 0x2.

Byte(s)	Type	Endian	Scaling
0–1	int16	Little-endian	$\div 10 = \text{degrees}$
2–7	(unused)	—	—

Parsed in firmware as:

```
currentSTA = (int16_t)((buf[0] | (buf[1] << 8)) / 10;
```

`currentSTA` is a signed 16-bit integer representing steering angle in tenths of a degree, divided to give degrees directly. Positive values are right-of-center; negative values are left-of-center (confirm the sign convention during initial vehicle testing).

4.2 Reading Strategy

The firmware polls the MCP2515 INT pin (active-low, GPIO 4) every loop. When the pin is low, a frame is waiting; the firmware reads and filters for ID 0x2. All other IDs are silently discarded (MCP_ANY mode accepts everything, but only 0x2 is stored).

Note

If you need to capture additional CAN signals (speed, gear position, etc.), add more `if (id == 0xXX)` branches inside `readCAN()`. No filter reconfiguration is needed because the firmware already accepts all IDs.

5 Steering PID Control Loop

5.1 Loop Structure

The control loop runs in the Arduino `loop()` function at the MCU's natural speed (no fixed rate delay). Loop timing is measured via `micros()` and passed as `dt` (seconds) into the PID. Values of `dt` outside (0, 1) are clamped to 0.01 s to prevent divide-by-zero or derivative spikes on startup.

5.2 Target Calculation and Smoothing

$$\text{targetSTA} = \text{steering} \times 250 \quad (\text{degrees})$$

A first-order IIR low-pass filter smooths the target before it enters the PID:

$$\text{smoothTarget}_k = \text{smoothTarget}_{k-1} + \alpha \cdot (\text{targetSTA} - \text{smoothTarget}_{k-1})$$

where $\alpha = 10 \cdot dt$. At a typical loop rate of $\sim 1 \text{ ms}$, $\alpha \approx 0.01$, giving a time constant of $\tau = 1/10 = 100 \text{ ms}$.

Note

The smoothing filter prevents abrupt steering commands (e.g., from a dropped and re-received packet with a large position jump) from immediately spiking the PID output. Reducing the coefficient (e.g., $5 \cdot dt$) makes steering feel “heavier”; increasing it makes it more responsive.

5.3 PID Gains

Gain	Variable	Value	Units
Proportional	kp	3.6	DAC counts / degree
Integral	ki	0.05	DAC counts / (degree·s)
Derivative	kd	0.1	DAC counts·s / degree

5.4 Anti-Windup and Clamping

The firmware implements two windup-prevention measures:

1. **Dead band reset:** if $|\text{error}| < 2\check{r}$, the integral is reset to zero. This prevents integrator drift from accumulated small errors near the target.
2. **Conditional integration:** the integral accumulates only when the proportional term alone has not saturated the output (i.e., $|k_p e + k_i I| < \text{maxTorque}$).
3. **Integral clamp:** the integral is hard-clamped to ± 250 DAC counts after each update.

The final output is clamped to ± 1500 DAC counts (**maxTorque**):

$$\text{torque} = \text{clamp}(k_p e + k_i I + k_d \dot{e}, -1500, +1500)$$

5.5 PID Tuning Guide

If you modify the vehicle or steering motor, you will likely need to re-tune. Follow this sequence on a **stationary vehicle**:

1. Set **ki** = 0 and **kd** = 0. Increase **kp** until steering reaches the target but oscillates.
2. Back off **kp** by $\sim 20\%$.
3. Increase **kd** until oscillation is damped.
4. Restore a small **ki** (start at 0.01) to eliminate steady-state error.
5. Validate at low speed before testing at higher speeds.

6 Network Architecture

6.1 Why a Relay Server?

The ESP32 connects to a mobile hotspot. Mobile carriers perform Network Address Translation (NAT): the ESP32 has a private IP on the hotspot LAN and a shared public IP assigned to the carrier's gateway. It is unreachable from the internet by direct address.

The relay server solves this via **UDP hole-punching**:

1. The ESP32 sends a **HEARTBEAT** packet to the OCI server every 2s. Because the packet originates from the ESP32, the hotspot's NAT table creates a mapping: *(public IP : ephemeral port) ↔ ESP32 private IP*.
2. The relay server records the ESP32's source address and port from the arriving **HEARTBEAT**.
3. When a **CMD** packet arrives from the operator laptop, the relay forwards it to that recorded address. The hotspot's NAT entry still exists (kept alive by the 2s heartbeat), so the packet is delivered.

6.2 Relay Server Logic

```
if message == "HEARTBEAT":
    car_address = addr          # register / refresh ESP32
                                endpoint
elif car_address and addr != car_address:
    sock.sendto(data, car_address) # forward operator packet to
                                car
```

Any packet originating *from* the car (`addr == car_address`) that is not a **HEARTBEAT** is silently dropped. This prevents the car from accidentally feeding data back to itself if it ever sends non-heartbeat traffic.

6.3 Packet Protocol

All packets are plain UTF-8 text, terminated with the UDP packet boundary (no newline needed).

Direction	Format	Notes
ESP32 → Server	HEARTBEAT	Sent every 2s
Laptop → Server	CMD;seq;steer;throttle;brake	Sent at 60 Hz
Server → ESP32	(forwarded CMD verbatim)	No modification

Field	Type	Range	Description
seq	uint32	0, 1, 2...	Monotonic counter; starts at 0
steer	float32	-1.0...+1.0	-1.0 = full left
throttle	float32	0.0...1.0	0.0 = released
brake	float32	0.0...1.0	0.0 = released

The ESP32 parses with `sscanf`:

```
sscanf(packetBuffer, "CMD;%lu;%f;%f;%f", &seq, &s, &t, &b)
```

There is no sequence-number validation or out-of-order rejection in the current firmware. If a late packet arrives it will overwrite the current control state.

6.4 Network Configuration

Parameter	Value
Relay server public IP	147.224.143.221
UDP port (all parties)	4210
Sender rate	60 Hz
Heartbeat interval	2 s

7 Software Architecture

7.1 g29_to_server_original.py (Operator Laptop)

Constant	Value / Description
STEER_AXIS_INDEX	0 — steering wheel axis
THROTTLE_AXIS_INDEX	2 — throttle pedal axis
BRAKE_AXIS_INDEX	3 — brake pedal axis
SERVER_IP	147.224.143.221
SERVER_PORT	4210
REFRESH_HZ	60

The G29 pedal axes report +1.0 when *released* and -1.0 when *fully pressed*. The sender normalizes and inverts:

```
def normalize_01_from_minus1_1(val: float) -> float:
    return max(0.0, min(1.0, (val + 1.0) / 2.0))

throttle = 1.0 - normalize_01_from_minus1_1(joystick.get_axis(2))
brake     = 1.0 - normalize_01_from_minus1_1(joystick.get_axis(3))
```

Step-by-step for the **throttle pedal fully pressed** (axis reports -1.0):

1. `normalize_01(-1.0) = ((-1) + 1)/2 = 0.0`

2. `throttle = 1.0 - 0.0 = 1.0`

Step-by-step for the **throttle pedal released** (axis reports **+1.0**):

1. `normalize_01(+1.0) = ((+1) + 1)/2 = 1.0`

2. `throttle = 1.0 - 1.0 = 0.0`

Steering is passed through *without* normalization; the G29 steering axis already reports **-1.0** (full left) to **+1.0** (full right).

7.2 relay_server.py (OCI Cloud VM)

Minimal single-socket server. Runs on 0.0.0.0:4210. The only state it maintains is `car_address` — the most recently seen HEARTBEAT source. All non-HEARTBEAT packets arriving from an address other than `car_address` are forwarded verbatim.

Design Decision

The relay server does not time out `car_address`. If the ESP32 reboots and obtains a new public IP/port from the hotspot, the first new HEARTBEAT will update the stored address. There is no danger of sending packets to a stale address indefinitely.

The server also prints a `DEBUG` line for every received packet. This is intentional for initial development but will flood the console during normal operation. To silence it, remove or comment out the two `print(f"DEBUG...")` lines.

7.3 esp32_car.ino (ESP32 on PCB)

7.3.0 Setup Sequence

1. `Serial.begin(115200)`
2. Initialize I²C buses (W1 on 21/22, W2 on 33/32, both at 400 kHz)
3. Initialize DAC 1 and DAC 2 (`begin(0x60, &W1/W2)`)
4. `initDACChannels()` — write initial DAC values + save to EEPROM
5. `initCAN()` — start SPI and MCP2515 at 500 kbps / 16 MHz
6. `initWiFi()` — connect to hotspot, start UDP listener on port 4210

7.3.0 Main Loop Sequence (per iteration)

1. Compute `dt` from `micros()` delta
2. Send HEARTBEAT if > 2000 ms since last heartbeat
3. `receiveUDP()` — parse latest `CMD` packet into `steering`, `throttle`, `brake`

4. `readCAN()` — update `currentSTA` from CAN ID 0x2
5. `updateBaseVoltages()` — recompute DAC baseline counts
6. Constrain `steering`, `throttle`, `brake`
7. Compute `targetSTA`, apply low-pass filter to `smoothTarget`
8. Compute `accelMag` and `brakeMag`
9. `calcTorque(smoothTarget, currentSTA, dt)` — run PID
10. `sendOutputs()` — write all six DAC values
11. Print debug to Serial every 100 ms

8 Known Limitations and Design Decisions

8.1 No Watchdog / Failsafe

△ WARNING

There is **no watchdog timer** in the current firmware. If the operator laptop stops sending packets (network drop, crash, Ctrl+C), the ESP32 continues targeting the **last received** steering angle, throttle, and brake values indefinitely. This means the car will continue accelerating and steering at its last commanded state until the safety driver intervenes.

A simple fix is to timestamp the last received packet and reset all control variables to zero if no packet arrives within a timeout:

```
#define WATCHDOG_MS 200
unsigned long lastPacketMs = 0;

// In receiveUDP(), after successful parse:
lastPacketMs = millis();

// In loop(), after receiveUDP():
if (millis() - lastPacketMs > WATCHDOG_MS) {
    steering = 0.0f;
    throttle = 0.0f;
    brake     = 0.0f;
    integral  = 0.0f; // reset PID integrator
}
```

8.2 Open-Loop Throttle and Brake

Throttle and brake are driven open-loop: there is no speed sensor feedback. The DAC offsets were determined empirically on the test vehicle. If the vehicle or actuators change, the scaling

factors (300 for throttle, 900 for brake) will need to be re-calibrated.

8.3 EEPROM Initialization Mismatch

`initDACChannels()` writes initial values to EEPROM that do *not* match the runtime channel assignments. For example, it writes steering midpoint values to DAC 1 channels C/D, but during runtime DAC 1 Ch C/D drive the brake. This is harmless because the firmware immediately overwrites all channels via `fastWrite()` on the first loop iteration. However, if the ESP32 powers up without running `loop()` (e.g., it crashes in `setup()`), the EEPROM-default outputs may be unexpected. A future revision should align the EEPROM values with the runtime assignments.

8.4 DEBUG Prints in `relay_server.py`

The relay server prints a debug line for *every* received UDP packet:

```
DEBUG - Received: 'CMD;0;0.0000;0.0000;0.0000' from ('1.2.3.4',  
54321)
```

At 60 Hz this generates 60 lines per second and will fill terminal scrollbar quickly. Remove the two `print(f"DEBUG...")` lines once the system is confirmed working.

8.5 Relay Server Has No Timeout on Car Address

If the car physically moves out of hotspot range, `car_address` in the relay server is never cleared. CMD packets from the operator will be silently forwarded to a stale address with no error shown. The operator only notices because the car stops responding. A future revision should print a warning if no HEARTBEAT is received within ~10s.

9 Modification Guide

9.1 Changing WiFi Credentials

In `esp32_car.ino`, update and re-flash:

```
const char* ssid      = "NEW_SSID";  
const char* password  = "NEW_PASSWORD";
```

9.2 Changing the Relay Server IP or Port

In `esp32_car.ino`:

```
const char* serverIP = "NEW.IP.ADDRESS";  
const int   serverPort = NEW_PORT;
```

In `g29_to_server_original.py`:

```
SERVER_IP    = "NEW.IP.ADDRESS"  
SERVER_PORT = NEW_PORT
```

Both files must be updated. The relay server binds to 0.0.0.0 and uses whatever port is defined in `UDP_PORT`, so update that constant too if you change the port.

9.3 Adjusting Throttle and Brake Scaling

The scaling factors are literal constants in `loop()`:

```
accelMag = pedalMagnitude(appliedThrottle, 300.0f); // max  
          offset = 300  
brakeMag = pedalMagnitude(appliedBrake,      900.0f); // max  
          offset = 900
```

Increase the second argument to increase the maximum actuator travel. Verify that the resulting DAC count (`accel1Base + accelMag`) does not exceed 4095 at full input, or the `constrain()` in `sendOutputs()` will clip it.

9.4 Reading Additional CAN Signals

Add cases inside `readCAN()`:

```
if (id == 0x2) {  
    currentSTA = (int16_t)((buf[0]) | (buf[1] << 8)) / 10;  
}  
// Example: read vehicle speed from a different CAN ID  
if (id == 0x1B4) {  
    vehicleSpeed = (buf[0] | (buf[1] << 8)) * 0.01f; // example  
               scaling  
}
```

No filter configuration is needed because the MCP2515 is in `MCP_ANY` mode.

9.5 Adding the Watchdog

See Section 9.1 for the recommended code snippet. After adding it, test by unplugging the laptop's network cable with the car stationary and confirming that all actuators return to their idle state within 200 ms.